ML-Driven Trading on Kalshi BTC Binary Markets

Machine Learning Final Project

Todd Tan Aldrich Wang and Arush Guliani

The University of Chicago

March 2026

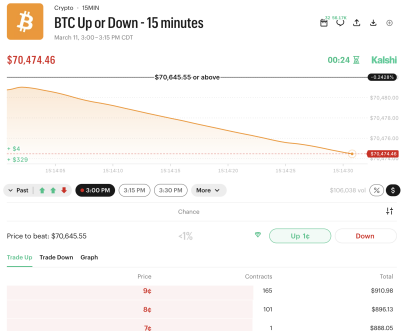
Problem Statement

The Exchange: Kalshi

- CFTC-regulated prediction market exchange
- Binary contracts: pays \$1 if event occurs, \$0 otherwise

BTC Up or Down — 15 Min

- New contract every 15 min: "Will BTC be $\geq$ \$X at expiry?"
- YES/NO sides trade as probabilities (cents); live bid/ask order book
- Settles at \$1 (YES) or \$0 (NO)



Data Sources

Kalshi API (REST & WebSocket)

- Top-of-book YES/NO: bid/ask price & size
- Trade stats: count, volume, VWAP (last 60s)
- Quote-change event count per minute
- Frequency: **20-second snapshots**, real-time

Coinbase API

- BTC/USD spot price, **every 20 seconds**, real-time

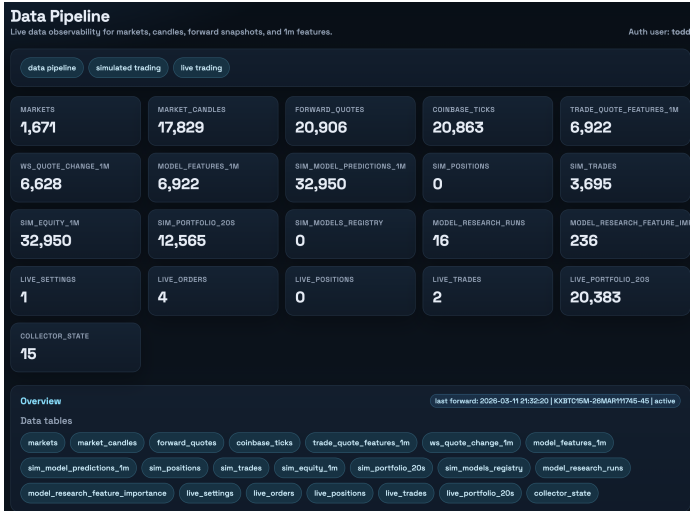
Collection & Storage

- Continuous collection: **Mar 6, 2026, 7:16 PM CST – present**
- MySQL on AWS RDS; 20s snapshots aggregated to 1-min bars
- EC2 workers: `forward_collector`, `coinbase_collector`, `feature_1m_collector`

Key Tables

- `forward_quotes` — 20s top-of-book
- `coinbase_ticks` — 20s BTC spot
- `model_features_1m` — 1-min ML feature table

Data Pipeline Dashboard



Feature Set

Order Book & Pricing

- `yes_bid/ask_dollars` — top-of-book prices
- `spread_dollars`, `mid_dollars`
- `orderbook_imbalance` — $(bid_sz - ask_sz)/(bid_sz + ask_sz)$

Contract Returns

- `mid_ret_1m`, `mid_ret_3m`

Trade Activity (last 60s)

- `trades_last_60s_count` — trade count
- `trades_last_60s_volume_fp` — dollar volume
- `trades_last_60s_vwap_yes` — VWAP
- `quote_change_events_60s` — book update frequency

Coinbase BTC Spot

- `cb_ret_1m`, `cb_ret_3m`, `cb_ret_5m`
- `cb_vol_5m` — 5-min realized volatility

Moneyness & Interactions

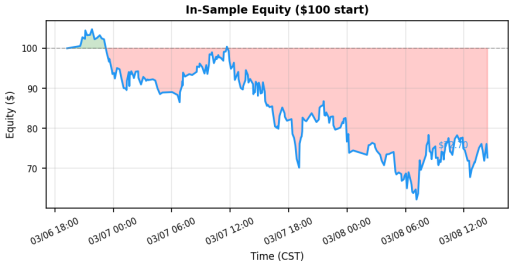
- `moneyness_abs/pct` — spot vs. strike
- `moneyness_vol_scaled` — $\div$ BTC vol
- `moneyness_x_sqrt_tte` — $\times\sqrt{TTE}$
- `moneyness_x_imbalance` — $\times$ imbalance
- `moneyness_x_qchg` — $\times$ quote changes

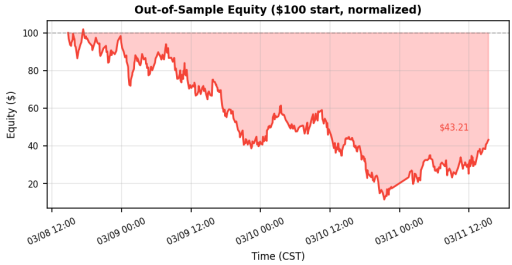
Model Baseline — Naive Logistic Regression

Label: $y_t = \mathbf{1}[\text{bid}_{t+1} > \text{ask}_t]$ **Features (13):** `bid`, `ask`, `spread`, `mid`, `imbalance`, `mid_ret_1m/3m`, `cb_ret_1m/3m`, `trades_count`, `quote_chg`, `tte`, `moneyness_pct`

Train (CST)	Mar 6 7:16 PM – Mar 8 6:16 AM	2,044 rows	In-sample
Test (CST)	Mar 8 6:16 AM – 2:47 PM	511 rows	In-sample holdout
OOS (CST)	Mar 8 2:47 PM – present	live	Forward test

Buy YES: $p_{\text{yes}} \geq 0.55$ & edge $> \$0.01$ **Sell YES:** $p_{\text{yes}} < 0.55$ or edge $\leq \$0.01$ (or TTE ≤ 60s)
Qty: 1–10 contracts (scaled by edge) **OOS PnL: –$56.79** / 474 trades / 49.7% win rate





Model #1 — Logistic Regression with L1 and Interactions

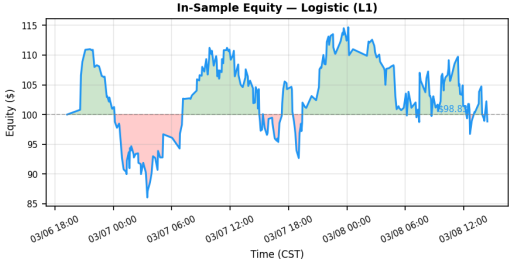
Label: $y_t = \mathbf{1}[\text{bid}_{t+1} > \text{ask}_t]$ **Features (22):** all baseline features + `cb_ret_5m`, `cb_vol_5m`, `moneyness_abs`, `moneyness_vol_scaled`, `moneyness_x_sqrt_tte`, `moneyness_x_imbalance`, `moneyness_x_trades_volume_fp`, `trades_vwap`

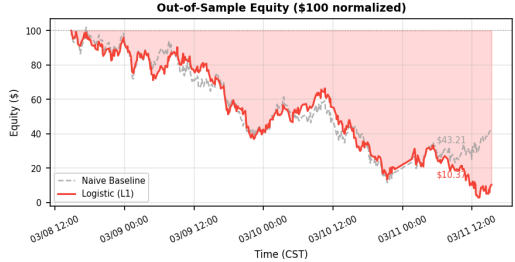
Regularization: L1 ($\lambda = 2.0$)

Train (CST)	Mar 6 7:16 PM – Mar 8 6:16 AM	2,044 rows	In-sample
Test (CST)	Mar 8 6:16 AM – 2:47 PM	511 rows	In-sample holdout
OOS (CST)	Mar 8 2:47 PM – present	live	Forward test

OOS PnL: –$89.63 ($100→$10.37) / 501 trades / 46.3% win rate

Comment: Added complexity is not justified; near-random signal (∼50/50) motivates non-linear models





Model #2 — Gradient Boosting

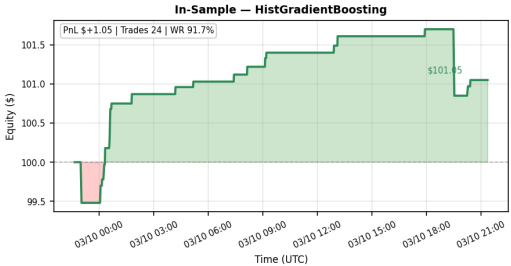
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (39):** all 22 production features + depth/pressure, flow context, return interactions, strike carry

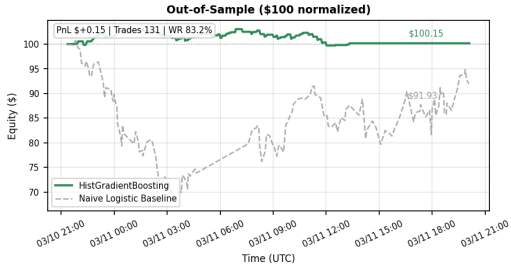
Params: `learning_rate=0.03, max_depth=4, max_iter=250, min_samples_leaf=25 l2=0.01, max_leaf_nodes=63`

Train	Mar 7 – Mar 9 (UTC)	3,545 rows	60% split
Validation	Mar 9 22:38 – Mar 10 21:21 (UTC)	1,182 rows	In-sample holdout
OOS (Test)	Mar 10 21:22 – Mar 11 20:06 (UTC)	1,182 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.55$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: +\$0.15 / 131 trades / 83.2% win rate **OOS AP:** 0.814, **AUC:** 0.810





Model #3 — Random Forest

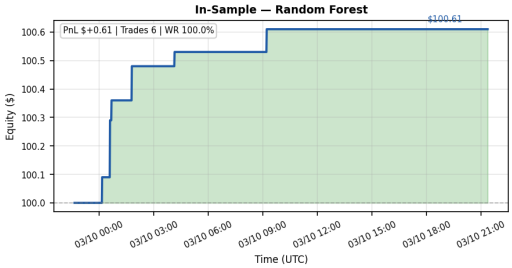
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (39):** all 22 production features + depth/pressure, flow context, return interactions, strike carry

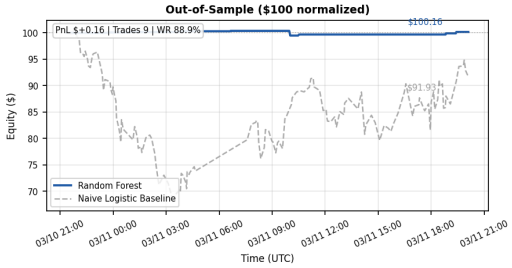
Params: `n_estimators=500, max_depth=10, min_samples_leaf=4, min_samples_split=12, max_features=sqrt, class_weight=balanced_subsample`

Train	Mar 7 – Mar 9 (UTC)	3,545 rows	60% split
Validation	Mar 9 22:38 – Mar 10 21:21 (UTC)	1,182 rows	In-sample holdout
OOS (Test)	Mar 10 21:22 – Mar 11 20:06 (UTC)	1,182 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.65$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: +\$0.16 / 9 trades / 88.9% win rate **OOS AP:** 0.848, **AUC:** 0.847





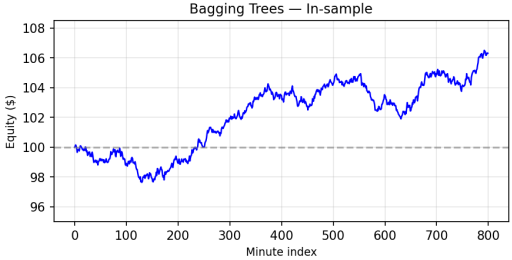
Model #4 — Bagging Trees

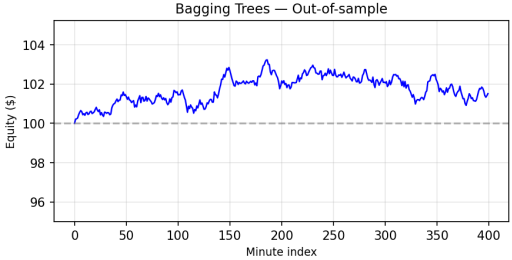
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (22):** same production features as logistic / random forest (order book, returns, moneyness, Coinbase)

Params: `BaggingClassifier: n_estimators=100, base DecisionTreeClassifier max_depth=6, min_samples_leaf=8, min_samples_split=16, max_samples=0.8, max_features=0.8`

Train	Mar 7 – Mar 9 (UTC)	3,545 rows	60% split
Validation	Mar 9 22:38 – Mar 10 21:21 (UTC)	1,182 rows	In-sample holdout
OOS (Test)	Mar 10 21:22 – Mar 11 20:06 (UTC)	1,182 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.55$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1
OOS PnL: TBD (run pending) **OOS AP:** –, **AUC:** –





Model #5 — AdaBoost (Boosted Trees)

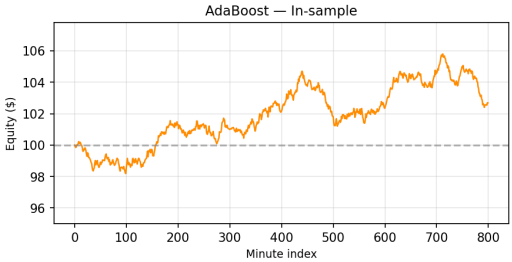
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (22):** same production features as logistic / random forest (order book, returns, moneyness, Coinbase)

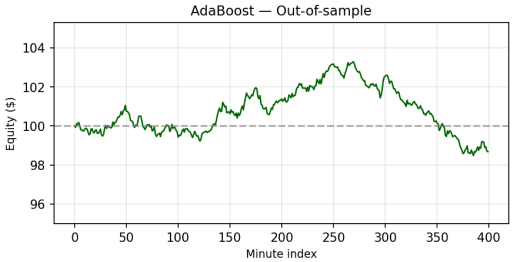
Params: `AdaBoostClassifier`: `n_estimators=100, learning_rate=0.1`, base `DecisionTreeClassifier` `max_depth=2, min_samples_leaf=4, min_samples_split=8`

Train	Mar 7 – Mar 9 (UTC)	3,545 rows	60% split
Validation	Mar 9 22:38 – Mar 10 21:21 (UTC)	1,182 rows	In-sample holdout
OOS (Test)	Mar 10 21:22 – Mar 11 20:06 (UTC)	1,182 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.55$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: TBD (run pending) **OOS AP:** –, **AUC:** –





Conclusion

What we did

- Built an end-to-end pipeline: Kalshi + Coinbase data, 20s→1-min features, MySQL, shared research and live trading engine
- Trained six models: naive logistic (baseline), L1 logistic, gradient boosting, random forest, bagging trees, AdaBoost — with the same train/validation/OOS splits and trading rules where applicable

Results (in-sample vs out-of-sample)

- Linear models (baseline, L1): negative OOS PnL; signal near 50/50 — motivated moving to non-linear models
- Tree ensembles (GBM, RF): positive OOS PnL, better calibration (AP/AUC ∼0.81–0.85); RF more selective (fewer trades, higher win rate)
- Bagging and AdaBoost: integrated into the research runner; same format and splits; backtest and plots TBD

Takeaways

- Feature design (moneyness, order book, BTC flow) and label choice (mid-price vs expiry-resolution) matter as much as model choice
- Risk and execution discipline (thresholds, position size, drawdown limits) are essential; avoid overfitting to noise

Next Steps

Models

- Complete bagging and AdaBoost backtests on the same train/validation/OOS splits; report in-sample and OOS PnL, win rate, AP, AUC in the same format as models 1–3
- Use feature importance (e.g., Shapley) to select a stable feature set and compare across all five ensemble types

Execution & risk

- Harden live engine: slippage and partial fills, expiry-boundary rules; keep drawdown and position limits

Monitoring

- Dashboards for calibration, PnL, and risk state; use the shared research/live stack to iterate on models and features without overfitting

Thank You

Questions & Discussion